İstanbul
Bilgi Üniversitesi

Artifical Neural Network Based Netlist Generator

by

Mehmet Beşenk, 118207067
Ahmet Emre Sertdemir, 117200037

Supervised by

Tuğba Dalyan
Dağhan Gökdel

Submitted to the
Faculty of Engineering and Natural Sciences
in partial fulfillment of the requirements for the

Bachelor of Science

in the
Department of Computer Engineering

Jan, 2022

Abstract

Simulation-ready representations of circuits are quite complicated and open to errors when the number of components in the circuit increases. Artificial intelligence-based methods gained more popularity in recent years to minimize human error and maximize accuracy. There have been many studies on the generation of Netlist-like representations with the vast improvement in neural networks. Most of those studies either only focus on the recognition of components in the circuit or only take basic circuits into account. This paper focuses on creating the required steps to make the generation of simulation-ready representations of circuits possible by using computer vision techniques which involves histogram oriented gradients, convolutional neural networks, optical character recognition, and image processing. Steps briefly consists of differentiating the components in the circuit. Detection of the nodes and deciding the how components interconnected to each other. After detecting the components, connecting them to the related nodes, the numerical values are extracted and assigned to the nearest circuit component. The model that has been trained with thousands of circuit images has more than 90% accuracy in detecting circuit components and the algorithms behind detection of lines provides a high accuracy.

Table of Contents

List of Figures

List of Abbreviations

YOLO	You Only Look Once
e.g.	Exempli gratia (Latin: for example)
MAP	Maximum a posteriori
MSE	Mean Squared Error
HOG	The Histogram of Oriented Gradients

1 Introduction

Nowadays, computer-aided applications and methods that will speed up the completion of routine calculation parts; allow more time and energy to be devoted to innovative research and development studies that make a difference in research projects, are developed with great effort. These methods typically rely on computers having the work carried out by the researcher manually, during the execution of a research. Artificial intelligence algorithms, which have recently become up-to-date thanks to the advances in computer hardware, are widely preferred in such applications. The main principle here is to teach computers the mechanized work that people do by spending time and energy, and thus ensure that the routine steps required at any stage of the research are done more quickly and accurately.

Traditionally, the first step in electronic circuit design is the selection of the circuit topology that will perform the required function. The designer can choose one of the various circuit topologies, taking into account the performance requirements and design constraints in the application using various equations, analytical calculations, and shortcuts for possible suitable topologies. It is possible to use SPICE and / or similar circuit simulation programs to roughly determine which possible topology is more appropriate for the desired solution. For the determination process, Netlists (Fig. 1) are used as code or text description of the electrical circuits.

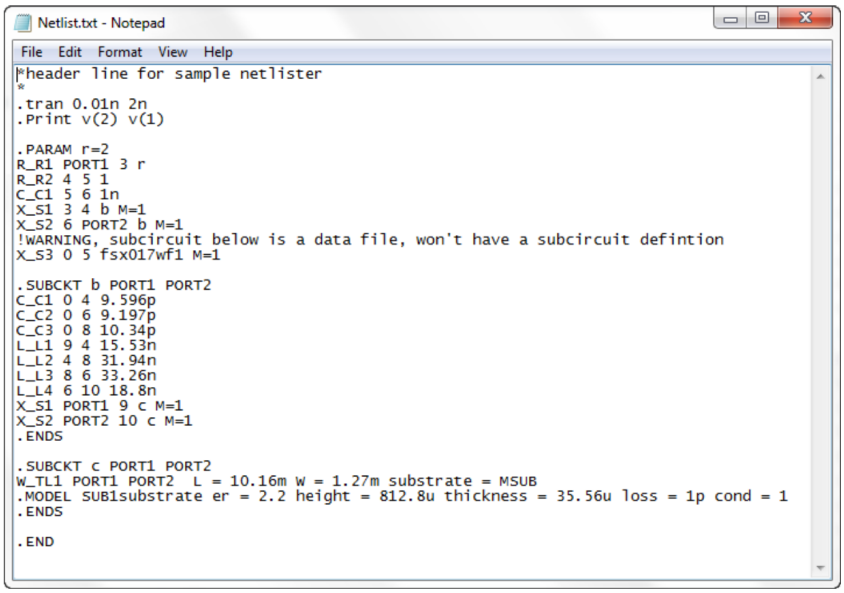

Figure 1: Example of the Netlist Code

The main problem in the validation phase of the topology options is to create the netlist of a circuit whose schematic is given effectively, quickly and accurately. The designer creates the netlist of a circuit whose scheme is given completely by hand, manually. Although the process of creating a connection list for low-element and relatively simple circuits does not seem difficult, for multi-element, complex integrated circuits, it can become a routine and error-prone process that becomes mechanized but still takes too much time. For this reason, designers often do the job of creating netlists manually during circuit design, often by mistake. In this context, within the scope of this proposed project, it is aimed to create a pre-design platform that will accelerate the electronic circuit design and analysis process of engineers, academics and students working on integrated circuit design, add insight and conceptual depth to the designer, and incorporate learning techniques. This platform, which is planned to be developed, will take any electronic circuit that the users are working on in image format, analyze the topology and circuit elements of the circuit and the connections between these elements with the help of deep learning models and automatically create the netlist of the relevant circuit.

2 Related Works

When the publications in the literature are examined, it is seen that most of the studies for the detection and analysis of circuit elements focus only on the differentiation of circuit elements with different methods. From this point on, especially considering the variety of analog circuit designs and the wide range of electronic circuit elements that can be used, it is obvious that the task of deriving the automatic circuit netlist and making the initial calculations is extremely necessary, but it is also a difficult research problem to realize. About circuit element detection research, in [1], the k-nearest neighborhood algorithm has been trained using 3500 circuit element photographs, and the developed algorithm detects the circuit elements with 93 accuracy. In [2], which is a similar study, researchers use 100 hand-drawn electrical circuit photographs to identify circuit elements with the Hidden Markov Model algorithm with 87 success rate. In [3], where circuit element detection was made by using 2D dynamic programming technique, a dataset with 130 images was used and 10 different circuit elements were 2 detected with a success rate of over 90 In another study [4], 92 of nodes and 86 of circuit elements were detected. Again, in [5], hand drawn circuit element detection was made with the help of an algorithm named Histogram of Oriented Gradients (HOG) that estimates according to the density distribution.

In [6], the researchers used a rule-based pattern recognition algorithm and developed a SPICE plug-in that can analyze the connections between them at a certain rate by recognizing the elements of the circuit with the generated algorithm. It has been observed that the accuracy rate of the algorithm varies between 66-84 in different situations. In [7], the researchers developed an algorithm that detects hand drawn circuit elements with an average of 85 success by applying morphological processes. However, in this study, as in the other studies mentioned above, it cannot be determined how the circuit elements are connected to each other. In [8], researchers have developed an algorithm that detects circuit elements in hand drawn circuit schematics with an average of 78 success using image processing techniques, and the algorithm in question cannot determine how these elements are interconnected. In addition to these studies, deep learning algorithms have been used in some studies on circuit element detection. For example, in [9], analysis of hand drawn circuits using deep learning architecture was made, a system based on Convolutional Neural Networks (CNN) was designed, circuit element estimation was made with a rate of 95, but as a result, the purpose of creating a connection list was not aimed. In a similar study using CNN architecture, [10], the researchers trained a deep learning model using a data set with 863 images consisting of hand drawn circuits. This model is designed to estimate only 4 different classes (resistor, capacitor, inductor and current source) and its accuracy rate is 84.41. In another study utilizing deep learning algorithms [11], only the circuit elements were determined, and no contribution was made to the analysis of the circuit or the Netlist. In another study [12], researchers have developed an algorithm that can successfully detect digital circuit elements and the structure of the circuit in the range of 75 -95 using deep learning algorithms. In the literature, it is seen that artificial neural networks are frequently used in object detection applications. In a study [13], a CNN-based system was used to identify real electronic circuit elements and a success rate of 93.4 was achieved. In [14], where the house numbers in street photographs were determined, a hybrid structure was created by using Convolutional Neural Networks (CNN) and the Hidden Markov Model together, and the performance rate of the Gaussian Mixture 3

Model output was tried to be increased. At the end of the study, a performance increase of 47.2 was observed. In [15-16], researchers have developed an object detection algorithm that detects defects in printed circuit board (PCB) production from the images of the cards with 90 and 98 success. As listed above, tools have been developed that can detect circuit elements with certain accuracy. However, according to the literature research, a tool that automatically generates the connection list of circuits has not been developed or reported in the literature yet. But in the platform planned to be developed

in this project, the Netlist of the circuit that the user should prepare by hand will be produced automatically by detecting the different circuit components and the connections.

3 Methodology

3.1 Dataset

1300+ images were collected and labeled with 19 different circuit components. With 7.4 labels per image average, the dataset has almost 10,000 labeled components. Dataset consist of mostly following components; Resistor,Capacitor, DC Source, Diode, BJT, Inductor, Voltage Source, Current Source, Battery, Opamp and Mosfet. Circuit images derived from both hand written and digital formats.

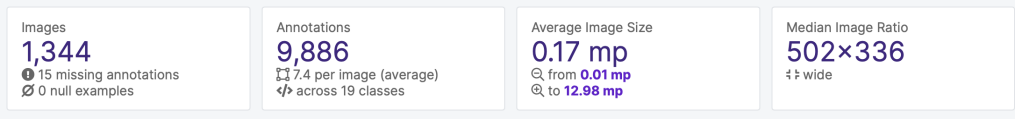

Figure 2: Values of the Dataset

Figure 3: Overview of the Dataset

3.2 Object Detection

Object detection is a computer vision technique for locating instances of objects in images or videos. Object detection algorithms typically leverage machine learning or deep learning to produce meaningful results. When humans look at images or video, we can recognize and locate objects of interest within a matter of moments. The goal of object detection is to replicate this intelligence using a computer.

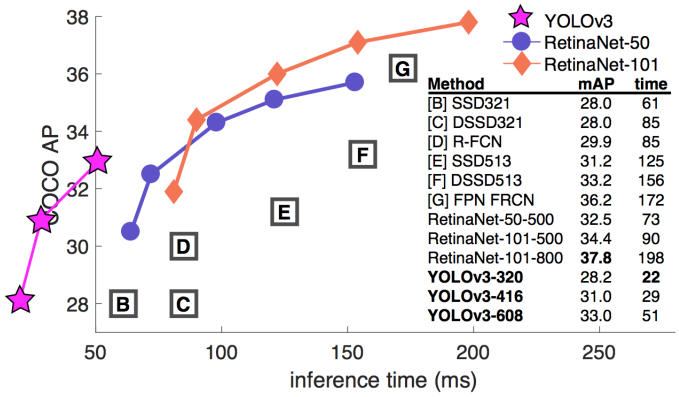

Figure 4: Comparsion of the Object Detection Models

In order to analyze the circuit all components in the image must be detected. There are many different object detection algorithms but YOLOv5 stands out for its ease of use, performance, and speed. YOLOv5 was able to perform with over 90 accuracy on the dataset with higher detection runtime in comparison with other object detection models.

Figure 5: Detection of the circuit elements

3.3 Node Detection

Nodes are the points connecting two different circuit elements. They are used to represent the points the circuit elements are connected to, thus playing a crucial role in netlist generation. To find the nodes, the lines in the circuit must be detected first. Hough Transform is an effective way to extract features in an image. The widely used application of Hough Transform is line detection.

After detecting all the lines in the image using Hough Transform, each line is separated into vertical and horizontal lines. This is done by comparing the values that have been obtained from the Hough Transform. After applying the transform on the image each detected line is represented by rho and theta values. $r = x\cos\theta + y\sin\theta$, Rho (r) is the distance to the closest point on the line from the origin point and theta is the angle between that line and the x-axis. With these values, the equations of the lines in the circuit are obtained. This representation of the lines does not give the starting and ending points of the lines. We can obtain the nodes by finding all the intersection points of the lines. To get the intersection points we must first find which lines are horizontal and which lines are vertical lines. This is done by comparing the theta values of each of the lines. If $3\pi/4 < \theta < \pi$ the line is vertical, if not the line is horizontal. After this separation, the intersection points are acquired. Since we got more lines than we actually have in the circuit the intersection points are also more than the actual number. This is solved by clustering the points by using scipy.cluster library.

After clustering the points we get the actual nodes. But since the intersection points of the lines can be anywhere we verify the node's authenticity by a number of processes such as nodes must be outside of all the bounding boxes detected by the model, every node must have at least 2 outgoing lines from it, etc.

```
def h_v_lines(lines):
    h_lines, v_lines = [], []
    for rho, theta in lines:
        if theta < np.pi / 4 or theta > np.pi - np.pi / 4:
            v_lines.append([rho, theta])
        else:
            h_lines.append([rho, theta])
    return h_lines, v_lines

def line_intersections(h_lines, v_lines):
    points = []
    for r_h, t_h in h_lines:
        for r_v, t_v in v_lines:
            a = np.array([[np.cos(t_h), np.sin(t_h)], [np.cos(
```

```
    t_v), np.sin(t_v)]])
15              b = np.array([r_h, r_v])
16              inter_point = np.linalg.solve(a, b)
17              points.append(inter_point)
18      return np.array(points)
19
20  def cluster_points(points):
21      dists = spatial.distance.pdist(points)
22      single_linkage = cluster.hierarchy.single(dists)
23      flat_clusters = cluster.hierarchy.fcluster(single_linkage,
    15, 'distance')
24      cluster_dict = defaultdict(list)
25      for i in range(len(flat_clusters)):
26          cluster_dict[flat_clusters[i]].append(points[i])
27      cluster_values = cluster_dict.values()
28      clusters = map(lambda arr: (np.mean(np.array(arr)[:, 0]), np
    .mean(np.array(arr)[:, 1])), cluster_values)
29      return sorted(list(clusters), key=lambda k: [k[1], k[0]])
```

Code 1: An listing example

Algorithm 1 Finding Nodes from the Circuit Images

```
function FIND_NODES

    lines = find_lines(image)
    horizontal_lines , vertical_lines = h_v_lines(lines)
    points = line_intersections(horizontal_lines, vertical_lines)

    return cluster_points(points)
end function
```

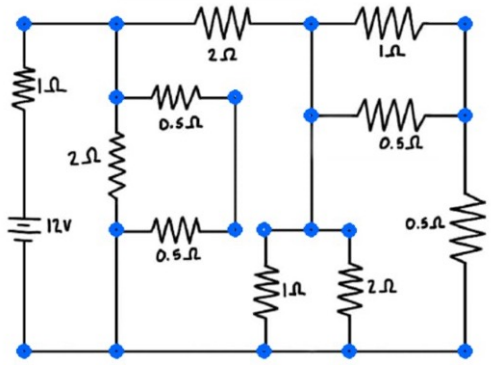

Figure 6: Detection of the nodes in the circuit

3.4 Optical Character Recognition

Optical character recognition (OCR) is the electronic or mechanical conversion of images of typed, handwritten, or printed text into machine-encoded text, whether from a scanned document, a photo of a document, a scene photo (for example, text on signs and billboards in a landscape photo), or subtitle text superimposed on an image (for example: from a television broadcast). It is a common method of digitizing printed texts so that they can be electronically edited, searched, stored more compactly, displayed on-line, and used in machine processes such as cognitive computing. It is widely used as a form of data entry from printed paper data records – whether passport documents, invoices, bank statements, computerized receipts, business cards, mail, printouts of static-data, or any suitable documentation.

Optical character recognition part provides the numeric values in the images. After extracting the texts, the values are attached to the corresponding components. This will be the last step of the nestlist generation process.

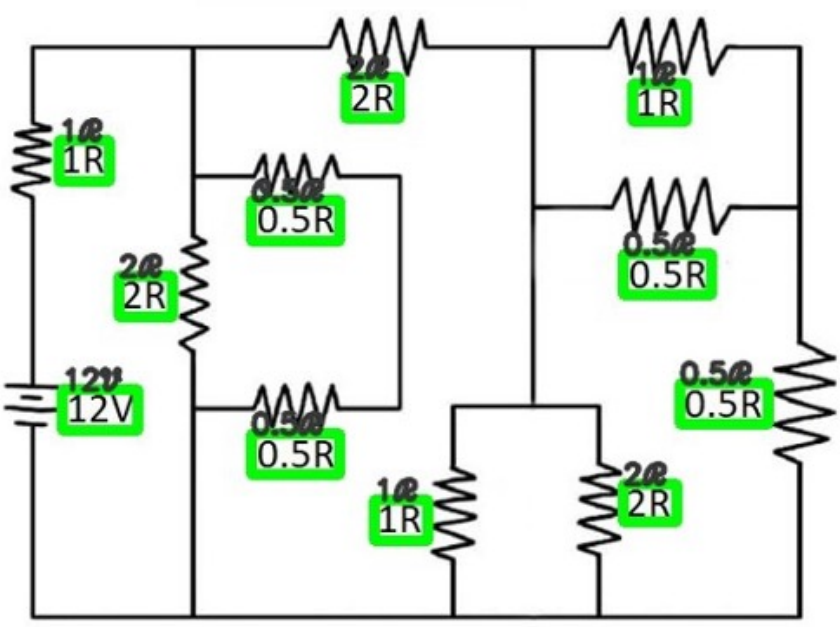

Figure 7: Optical Character Recognition from Circuit

3.5 Netlist Generation

Nodes that are in the same voltage level were named the same to indicate they are the same nodes. All the circuit elements were connected to the corresponding nodes using simple mathematical formulas. After achieving all the mentioned detections and connections the netlist representation was generated.

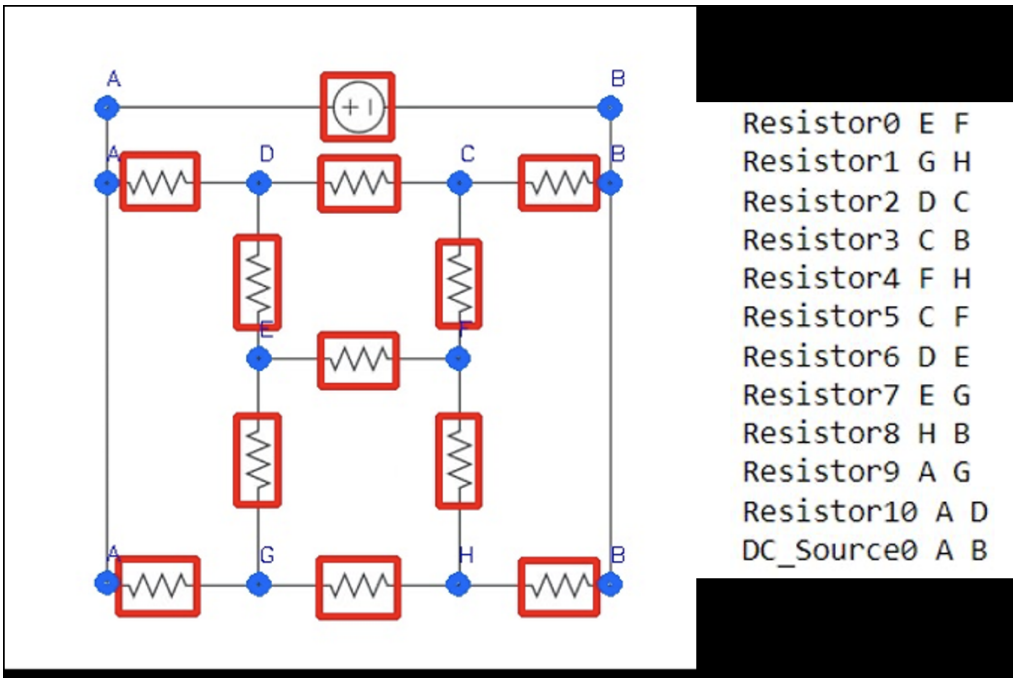

Figure 8: Generated netlist of the circuit

4 Referencing

References

[1] Huoming Zhang , Lixing Shao, 'Research on K nearest neighbor identification of hand-drawn circuit diagram' , Journal of Physics: Conference Series (2019)

[2] ZHANG Yingmin , VIARD-GAUDIN Christian , WU Liming , 'An Online Hand-Drawn Electric Circuit Diagram Recognition System Using Hidden Markov Models' , International Symposium on Information Science and Engineering (2008)

[3] Feng, G., Viard-Gaudin, C., Sun, Z. . On-line hand-drawn electric circuit diagram recognition using 2D dynamic programming. Pattern Recognition, 42(12), 3215-3223 (2009).

[4] Angadi, M., Naika, R. L. (2014). Handwritten circuit schematic detection and simulation using computer vision approach. International Journal of Computer Science and Mobile Computing, 3(6), 754-761.

[5] Edwards, B., Chandran, V. Machine recognition of hand-drawn circuit diagrams. In 2000 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No. 00CH37100) (Vol. 6, pp. 3618-3621). IEEE. (2000, June)

[6] Soham Roy, Archan Bhattacharya, Navonil Sarkar, Samir Malakar, Ram Sarkar, 'Offline hand-drawn circuit component recognition using texture and shape-based features', Multimedia Tools and Applications (2020)

[7] Gennaria Leslie, Kara Levent Burak ,Stahovich Thomas F. , Shimada Kenji , 'Combining geometry and domain knowledge to interpret hand-drawn diagrams' , Computers Graphics 29 (2005)

[8] Archan Bhattacharya, Soham Roy, Navonil Sarkar, Samir Malakar, Ram Sarkar, 'Circuit Component Detection in Offline Handdrawn Electrical/-Electronic Circuit Diagram', IEEE Calcutta Conference (2020)

[9] Haiyan Wang, Tianhong Pan, Mian Khuram Ahsan, 'Hand-drawn electronic component recognition using deep learning algorithm', International Journal of Computer Applications in Technology (2020)

[10] Mihriban Günay, Murat Köseoğlu, and Özal Yıldırım. "Classification of Hand-Drawn Basic Circuit Components Using Convolutional Neural Networks." In 2020 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), pp. 1-5. IEEE, 2020.

[11] Rabbania Mahdi , Khoshkangini Reza, Nagendraswamy H.S., Conti Mauro , 'Hand Drawn Optical Circuit Recognition' , 7th International conference on Intelligent Human Computer Interaction (2015)

[12] Altun Oğuz, Nooruldeen Orhan, 'Stroke-Based Recognition of Online Hand-Drawn Sketches of Arrow-Connected Diagrams and Digital Logic Circuit Diagrams', Hindawi Scientific Programming (2019)

[13] Ju, Y., Chen, Y. . An Ultra Lightweight CNN for Low Resource Circuit Component Recognition. arXiv preprint arXiv:2010.00505.(2020)

[14] Guo, Q., Wang, F., Lei, J., Tu, D., Li, G. Convolutional feature learning and Hybrid CNN-HMM for scene number recognition. Neurocomputing, 184, 78-90.(2016)

[15] Runwei Ding, Linhui Dai, Guangpeng Li, Hong Liu 'TDD-net: a tiny defect detection network for printed circuit boards', The Institution of Engineering and Technology Journals (2019)

[16] Hu Bing, Wang Jianhui, 'Detection of PCB Surface Defects with Improved Faster-RCNN and Feature Pyramid Network', IEEE Access (2020)

[17] CloudFactory. (n.d.). The Ultimate Guide to Data Labeling for Machine Learning. Retrieved January 19, 2021